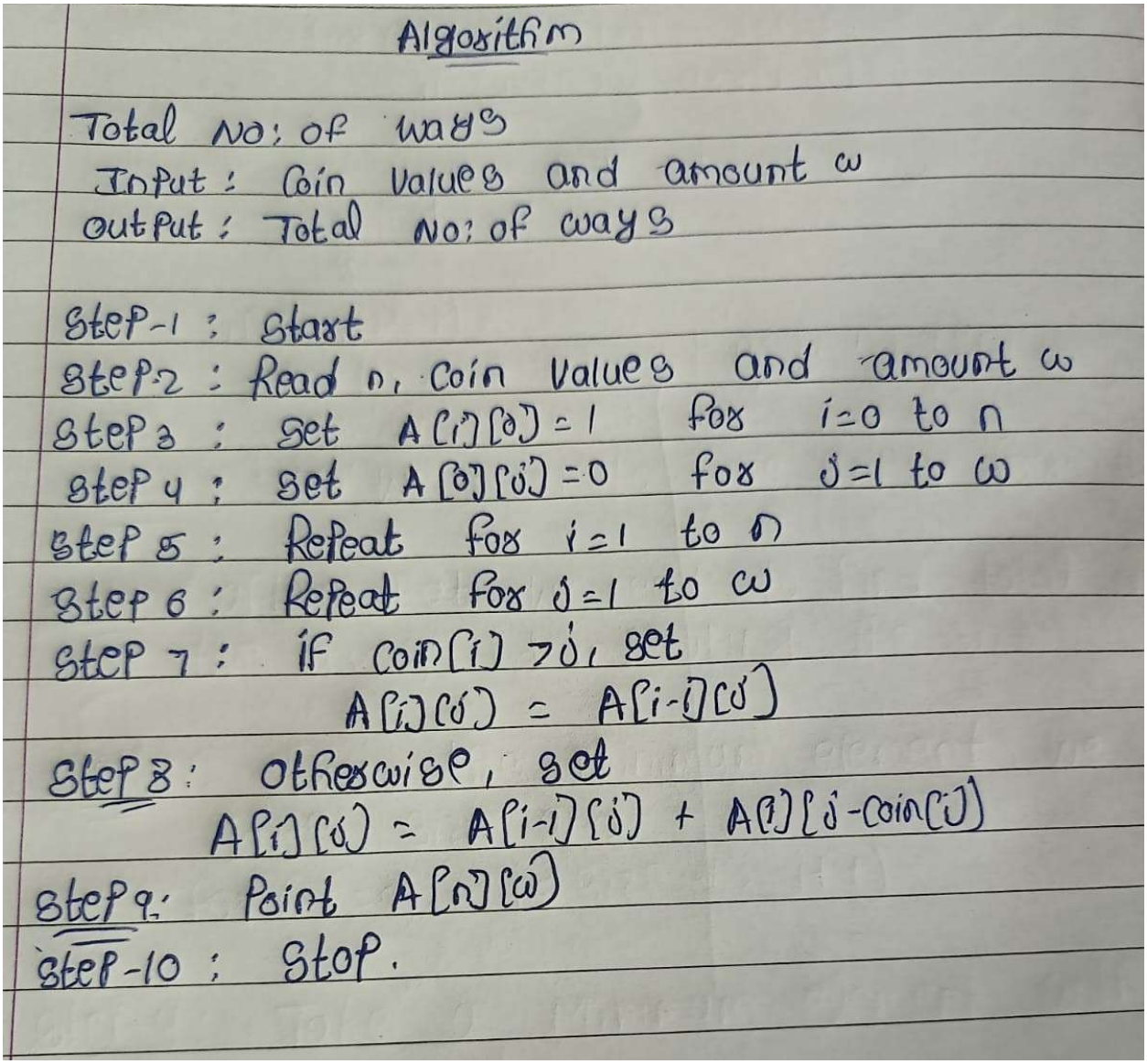
practical-7

Aim: Implementation of making a change problem using dynamic programming.

Making a change problem

Algorithm:



Algorithm

Total No: of ways
Input : Coin values and amount w
output : Total No: of ways

step-1 : start
step-2 : Read n, coin values and amount w
step 3 : set A[i][0] = 1 for i=0 to n
step 4 : set A[0][j] = 0 for j=1 to w
step 5 : Repeat for i=1 to n
step 6 : Repeat for j=1 to w
step 7 : if coin[i] >j, set
A[i][j] = A[i-1][j]
step 8 : otherwise, set
A[i][j] = A[i-1][j] + A[i][j-coin[j]
step 9 : Point A[n][w]
step-10 : Stop.

Program:

```
  #include <iostream>
using namespace std;

int main()
{
   int n, amount;

   cout << "Enter number of coins: ";
   cin >> n;

   int coin[n];

   cout << "Enter coin denominations: ";
   for (int i = 0; i < n; i++)
   {
      cin >> coin[i];
   }

   cout << "Enter amount: ";
   cin >> amount;

   int dp[amount + 1];

   dp[0] = 0;

   for (int i = 1; i <= amount; i++)
   {
      dp[i] = 32767;

      for (int j = 0; j < n; j++)
      {
         if (coin[j] <= i)
         {
            dp[i] = min(dp[i], dp[i - coin[j]] + 1);
         }
      }
   }

   cout << "Minimum number of coins required = "
```

```
        << dp[amount] << endl;

    return 0;
}
```

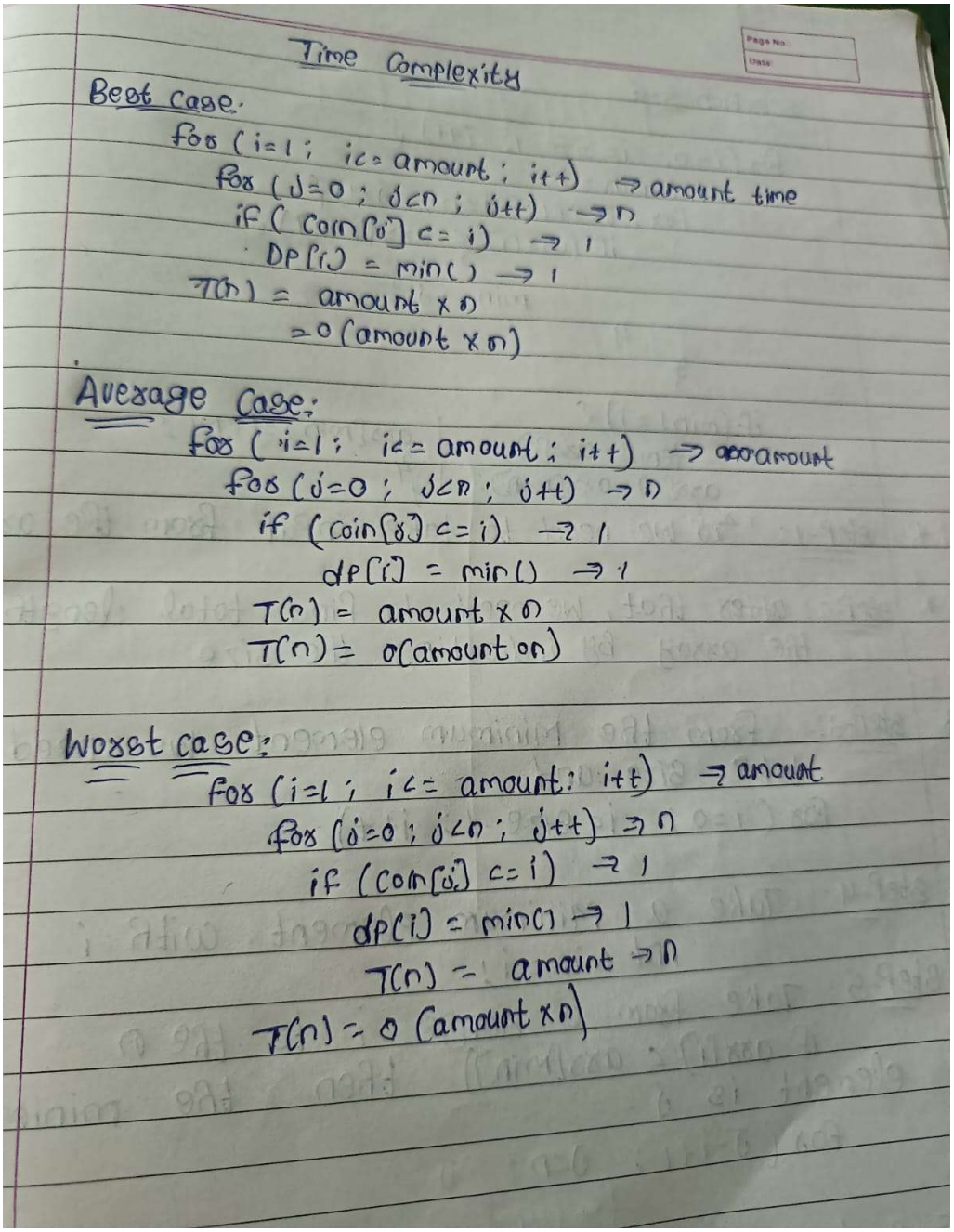
Output:

```
Output

Enter number of coins: 6
Enter coin denominations: 1 3 5 7 9 10
Enter amount: 17
Minimum number of coins required = 2


=== Code Execution Successful ===
```

Time Complexity:



Time Complexity

Best case:

for (i=1; i<= amount; i++) → amount time
  for (j=0; j<n; j++) → n
    if (coin[j] <= i) → 1
      DP[i] = min() → 1
T(n) = amount x n
= O(amount x n)

Average case:

for (i=1; i<= amount; i++) → amount
  for (j=0; j<n; j++) → n
    if (coin[j] <= i) → 1
      dp[i] = min() → 1
T(n) = amount x n
T(n) = O(amount x n)

Worst case:

for (i=1; i<= amount; i++) → amount
  for (j=0; j<n; j++) → n
    if (coin[j] <= i) → 1
      dp[i] = min() → 1
      T(n) = amount → n
T(n) = O(amount x n)

Conclusion:

The implementation and time analysis of the Making Change Problem using Dynamic Programming helps us understand its efficiency. The algorithm builds a DP table and considers different coin denominations to find the minimum number of coins required to make the given amount. It has a time complexity of O(n × amount) in the best, average, and worst cases. Therefore, Dynamic Programming provides an efficient and systematic solution for solving the Making Change Problem.